

第14章 类型系统进阶

建议学时:5-7 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 record:不可变数据载体的定义、紧凑构造器、组件访问
- 掌握 sealed 类:受限继承与 permits 的语义
- 掌握 switch 表达式与模式匹配:类型模式、守卫、穷尽性
- 理解"代数数据类型":sealed + record + 模式匹配的组合拳
- 掌握 Optional 的核心方法与链式编程
- 能对照 C 的 union/enum/结构体,说出类型安全上的进步
- 能综合运用上述特性写出无强制转换、无 null 检查的健壮代码

💡 从 C 到 Java:让"类型"替你挡住错误

前 13 章教你怎么写 Java;这一章教你怎么用类型设计程序。C 的类型系统是"宽松"的:union 里存什么全凭自觉,enum 会被悄悄转成 int,void* 强转后编译器闭嘴。Java 的现代类型系统反过来:把"约定"写进类型,让编译器在编译期替你检查。

record 是"值"的声明。C 里定义"一对坐标",你得手写 struct、初始化函数、打印函数;Java 的 `record Point(int x, int y)` 一行生成全部:构造器、getter、equals、hashCode、toString——而且是不可变的。不可变 = 线程安全(第9章)、可作 Map 的键(第10章)、无别名陷阱(第5章)。

sealed 类把"可能的取值"写进声明。C 的 enum 告诉你"有几种值",但拿它做状态机时,switch 忘了一个 case 编译器不吭声。sealed + switch 表达式让 编译器强制穷尽所有分支——漏一个 case 编译失败,这是 C 永远给不了的安全感。

Optional 把"可能没有"写进类型。C 的返回 -1/0/NULL 是注释约定,传错了没人拦;Java 的 `Optional<T>` 把"可能缺失"变成类型的一部分,逼你处理不存在的情况。

为什么重要:这三个特性(record/sealed/模式匹配)合称"代数数据类型"(Algebraic Data Type,ADT)在 Java 17 的落地,是函数式编程思想进入主流 Java 的标志。看懂它们,Java 代码在你眼里就分成两种:处理"数据"的代码与处理"行为"的代码,结构清晰得多。

学习顺序:record(§14.1)→ sealed(§14.2)→ switch 表达式与模式匹配 (§14.3)→ 组合成 ADT(§14.4)→ Optional(§14.5)。

📦 前置知识自查

- 第6章:继承、多态、抽象类(sealed 是继承的约束)
- 第10章:泛型(Optional<T>、record 泛型)
- 第9章:不可变对象与线程安全(record 的动机)
- C 语言:struct、union、enum、typedef 的习惯用法

🚀 本章学习路线

1. **第一阶段(约 1.5 小时):**§14.1 record,动手写 5 个 record,比较与手写 POJO 的差别
2. **第二阶段(约 1.5 小时):**§14.2 sealed + §14.3 switch 表达式与模式匹配,理解穷尽性检查
3. **第三阶段(约 2 小时):**§14.4 ADT 组合拳 + §14.5 Optional,完成章末实验(JSON 求值器)
4. **验收标准:**能用 sealed+record+switch 表达式写出无强制转换、无 null 判断、编译期穷尽的代码;能解释 Optional 链的每一步

本章知识导图

- §14.1 record:不可变数据载体
- §14.2 sealed 类:受限继承
- §14.3 switch 表达式与模式匹配
- §14.4 代数数据类型:sealed + record + 模式匹配
- §14.5 Optional:显式的"可能缺失"

§14.1 record:不可变数据载体

14.1.1 从冗长 POJO 到一行声明

C 程序员早已习惯写 struct + 一堆函数;Java 老式写法是 POJO(Plain Old Java Object):私有字段 + getter/setter + equals + hashCode + toString,六十行模板代码只为装两个数。record 把它压缩成一行,并且 免费送你正确性:字段 final、类 final、equals/hashCode/toString 按组件生成。

示例 14-1 record 基本用法

```
// 声明:Point 是一个 record,组件是 x 和 y
public record Point(int x, int y) {
    // 紧凑构造器:校验逻辑写在这里(不能给字段赋值)
    public Point {
        if (x < 0 || y < 0) {
            throw new IllegalArgumentException("坐标不能为负: " + x + ", " + y);
        }
    }

    // 额外方法照常加
    public double distance(Point o) {
        int dx = x - o.x, dy = y - o.y;
        return Math.sqrt(dx * dx + dy * dy);
    }
}

// 使用
public class RecordDemo {
    public static void main(String[] args) {
        Point p = new Point(3, 4);           // 全参构造器自动生成
        System.out.println(p.x());           // getter 不带 get 前缀:x()
        System.out.println(p);               // toString: Point[x=3, y=4]
        Point q = new Point(3, 4);
        System.out.println(p.equals(q));      // true:按组件比较
        System.out.println(p.hashCode() == q.hashCode()); // true
        // new Point(-1, 0) 抛 IllegalArgumentException(校验生效)
    }
}
```

示例 14-2 record 与类组合:局部 record

```
import java.util.ArrayList;
import java.util.List;

public class LocalRecordDemo {
    public static void main(String[] args) {
        // 局部 record:方法内部声明,立刻用
        record Price(int yuan, int fen) {
            int totalFen() { return yuan * 100 + fen; }
        }

        List<Price> prices = new ArrayList<>();
        prices.add(new Price(3, 50));
        prices.add(new Price(2, 99));

        int sum = 0;
        for (Price pr : prices) {
            sum += pr.totalFen();
        }
        System.out.println("总价: " + sum + " 分 = " + (sum / 100) + " 元 "
            + (sum % 100) + " 分");
    }
}
```

★ 重点:record 是不可变的"值",不是可变"实体"

record 适合"数据快照":坐标、金额、订单行、配置项。不适合需要演化状态的实体(计数器、连接池)——那种还是用普通类。判断标准: "这个对象换字段值是否代表另一个对象?"若是,用 record;若否,用类。C 的 struct 是可变副本,record 是 immutable value——多线程下免费安全(呼应第9章)。

⚠ 易错点 1:record 的字段是 final,没有 setter,组件不可重新赋值

record 隐含的"全字段构造器"是唯一的赋值通道;紧凑构造器里给 x 赋值会编译错误。想派生新值,用"with"方法返回新 record: `public Point withX(int nx) { return new Point(nx, y); }`——这正是不变式编程的姿势。另外注意:record 不能 extends 其他类(已隐式 extends java.lang.Record),但可以实现接口。

§14.2 sealed 类:受限继承

14.2.1 谁可以继承我,白纸黑字

普通继承是开放的:任何类都能 extends 你的类。sealed 把继承名单写进声明: "只有这些类能继承我,且它们必须同模块/同包,且必须标注 final/sealed/non-sealed"。C 语言里最接近的心智是 enum + 显式分发,但 sealed 是编译期强制的。

示例 14-3 sealed 类:形状体系

```
// 父类封闭:只允许 Circle、Rect、Triangle 继承
public sealed interface Shape
    permits Circle, Rect, Triangle {
    double area();
}

// 直接子类必须标注 final / sealed / non-sealed 之一
public final class Circle implements Shape {
    private final double r;
    public Circle(double r) { this.r = r; }
    public double area() { return Math.PI * r * r; }
}

public final class Rect implements Shape {
    private final double w, h;
    public Rect(double w, double h) { this.w = w; this.h = h; }
    public double area() { return w * h; }
}

// non-sealed:继续开放给第三方
public non-sealed class Triangle implements Shape {
    private final double base, height;
    public Triangle(double base, double height) {
        this.base = base; this.height = height;
    }
    public double area() { return base * height / 2; }
}
```

示例 14-4 精简版:sealed class 语法与"同一文件"技巧

```
// sealed class 与 interface 语法一致;permits 列表可省略(同文件时)
// 同一源文件中声明时,permits 可以省:
public sealed class Animal {
    // ...
}

final class Dog extends Animal { }           // 同文件:自动认可
final class Cat extends Animal { }

// 若子类在别的包,必须写 permits 且子类必须能访问父类:
// public sealed class Animal permits zoo.Dog, zoo.Cat { }
```

⚠ 易错点 2:sealed 的三个强制性条件

① 子类必须出现在 permits 列表(或与 sealed 类同文件);② 每个直接子类必须标 final / sealed / non-sealed 之一——忘记标注直接编译错误;③ 子类与父类须同模块,或同包(跨包需 permits 且子类可访问)。non-sealed 是"开个口子"给第三方扩展,一开口,穷尽性检查就失效——生产代码谨慎使用。

§14.3 switch 表达式与模式匹配

14.3.1 switch 表达式:有值、无 fall-through

老式 switch 是语句(C 程序员的老朋友,还有 break 陷阱);switch 表达式是"有返回值的表达式",每个分支自然结束、无需 break、类型统一,漏分支编译错误。

示例 14-5 switch 表达式基础

```

public class SwitchExprDemo {
    static String dayName(int day) {
        // 表达式形式:整个 switch 有值;箭头分支无需 break
        return switch (day) {
            case 1 -> "星期一";
            case 2 -> "星期二";
            case 3 -> "星期三";
            case 4 -> "星期四";
            case 5 -> "星期五";
            case 6, 7 -> "周末";
            default -> "非法日期: " + day;    // 必须给出兜底
        };
    }

    // yield:多语句分支返回
    static String describe(int day) {
        return switch (day) {
            case 1 -> {
                String s = "一";
                yield "周" + s;                // yield 带值返回
            }
            default -> "其他";
        };
    }

    public static void main(String[] args) {
        System.out.println(dayName(3));    // 星期三
        System.out.println(dayName(7));    // 周末
        System.out.println(describe(1));    // 周一
    }
}

```

14.3.2 模式匹配:对类型做 switch,而不是对值

示例 14-6 instanceof 模式匹配与 switch 模式标签

```

// instanceof 模式匹配:类型判断 + 自动强转,一步到位
public static String describe(Object o) {
    if (o instanceof String s) {
        return "字符串,长度 " + s.length();
    } else if (o instanceof Integer i) {
        return "整数 " + i;
    } else {
        return "其他类型";
    }
}

// switch 模式标签:对"类型"分支
public static String classify(Object o) {
    return switch (o) {
        case String s -> "字符串: " + s;
        case Integer i -> "整数: " + i;
        case Long l -> "长整: " + l;
        case null -> "是 null";                // 显式处理 null
        default -> "未知类型";
    };
}

// 守卫(guard):模式 + 条件过滤
public static String scoreLevel(Object o) {
    return switch (o) {
        case Integer i when i >= 90 -> "优秀";
        case Integer i when i >= 60 -> "及格";
        case Integer i -> "不及格";
        default -> "不是成绩";
    };
}

```

★ 重点:模式匹配消灭"强转 + null 判断"

老代码 `if (o instanceof String) { String s = (String) o; ... }` 三步;新代码一步 `case String s`。且: `switch` 表达式对 `sealed` 类型穷尽时,可以省略 `default`——编译器知道你列全了。C 的强转是"我说了算",Java 的模式匹配是"编译器验证我说得对"。

§14.4 代数数据类型:sealed + record + 模式匹配

14.4.1 组合拳:安全的"有穷取值"建模

三个特性合起来,表达力质变: `sealed` 限定取值集合(枚举维度),`record` 承载每个取值的载荷(积维度),`switch` 表达式穷尽分发(处理维度)。C 里这个模型是 `union + enum + 手写 switch`——漏 `case`、转错字段都是运行时炸弹;Java 版本全部编译期检查。

示例 14-7 表达式树:ADT 的经典例子

```
// 表达式:数字 / 加法 / 乘法,三种"变体",每个带自己的数据
public sealed interface Expr permits Num, Add, Mul {
}

public record Num(int value) implements Expr { }

public record Add(Expr left, Expr right) implements Expr { }

public record Mul(Expr left, Expr right) implements Expr { }

// 求值器:穷尽分发,编译器保证不漏分支
public final class Evaluator {
    public static int eval(Expr e) {
        return switch (e) {
            case Num n -> n.value(); // 拆 record
            case Add a -> eval(a.left()) + eval(a.right());
            case Mul m -> eval(m.left()) * eval(m.right());
            // 没有 default!编译器知道 sealed 只有三种
        };
    }
}

public static void main(String[] args) {
    // (1 + 2) * 3
    Expr e = new Mul(new Add(new Num(1), new Num(2)), new Num(3));
    System.out.println(eval(e)); // 9
}
```

维度	C 的写法	Java 17 的写法	安全差异
变体	enum + union	sealed interface	漏变体:编译报错
载荷	union 成员(自觉)	record 组件(强制)	读错字段:编译报错
分发	switch + 强转	switch 模式匹配	类型不符:分支不进,不是崩溃
缺失值	NULL / -1 约定	Optional<T>	不处理则编译警告/必须处理

§14.5 Optional:显式的"可能缺失"

14.5.1 为什么消灭 null

C 的 NULL 是"约定";Java 的 null 是十亿美金错误(第5章 NPE)。 `Optional<T>` 是一个包装盒:里面要么有 T,要么空——"可能没有"从注释变成类型。注意:Optional 是给"返回值"用的,不是给字段和参数用的。

示例 14-8 Optional 核心链式操作

```
import java.util.Optional;

public class OptionalDemo {
    // 数据库风格的查找:可能没有用户
    static Optional<String> findNickname(int id) {
        if (id == 1) {
            return Optional.of("小明");
        }
        return Optional.empty();
    }

    public static void main(String[] args) {
        Optional<String> nick = findNickname(1);

        // orElse:没有就给默认值
        System.out.println(nick.orElse("匿名用户"));

        // orElseGet:延迟求值(推荐,不计算就不用)
        System.out.println(findNickname(2).orElseGet(() -> "游客" + System.nanoTime()));

        // orElseThrow:没有就抛异常(语义最清晰)
        String name = findNickname(1)
            .orElseThrow(() -> new IllegalStateException("用户不存在"));

        // map:有值才转换,没有还是空(链式)
        Optional<Integer> len = nick.map(String::length);

        // filter:有值且满足条件才保留
        Optional<String> longName = nick.filter(s -> s.length() > 3);

        // flatMap:转换结果本身是 Optional,避免嵌套
        Optional<String> upper = nick.flatMap(
            s -> Optional.of(s.toUpperCase()));

        // ifPresent:有值才做副作用
        nick.ifPresent(s -> System.out.println("欢迎回来, " + s));

        // 三件套终端操作:map/filter/flatMap 是中间,orElse* 是终端
        String result = findNickname(99)
            .map(String::toUpperCase)
            .filter(s -> !s.isEmpty())
            .orElse("N/A");
        System.out.println(result);           // N/A
    }
}
```

⚠ 易错点 3:Optional 的四个使用红线

① 不做字段(序列化、性能、语义都说不通);② 不做方法参数(调用方仍然可能传 null,还多一层包装——不如直接判空);③ 不用 get() 裸取(空盒子 get 抛 NoSuchElementException,与 NPE 无异,用 orElse/orElseThrow);④ 不把集合包进 Optional(空集合本身就能表达"没有",Optional.of(List.of()) 是双重包装)。记住:Optional 解决的是"单个值可能缺失",不是通用 null 替代品。

🚀 工程建议:现代 Java 代码风格速览

1. 数据用 record,行为用类方法:DTO、返回值、事件载荷一律 record
2. 状态机与协议用 sealed + switch 表达式:漏分支编译不过,重构时编译器当哨兵
3. 查找类 API 返回 Optional<T>:调用方自然写出 orElse 链
4. 警惕 non-sealed:开了口子,穷尽性就没了,审慎

本章速查表

主题	要点
record	不可变数据载体;一行生成构造器/getter(x())/equals/hashCode/toString
紧凑构造器	校验逻辑所在;不能给组件赋值
record 限制	final、不可继承(隐式 extends Record)、可实现接口、可泛型
sealed	permit 列表限定继承;子类须 final/sealed/non-sealed
non-sealed	重新开放继承;代价是穷尽性失效
switch 表达式	箭头分支、无 break、有值;default 或穷尽时省略
yield	多语句分支用 yield 返回值
模式匹配	instanceof 模式 / switch 模式标签;自动绑定变量;免强转
守卫	case X x when 条件 -> 结果
ADT 组合	sealed(变体) + record(载荷) + switch(分发) = 编译期穷尽
Optional 中间操作	map / filter / flatMap;有值才继续,空则保持空
Optional 终端操作	orElse / orElseGet / orElseThrow / ifPresent
Optional 红线	不做字段、不做参数、不 get()、不包集合

最小必会清单

- 能写 record 声明、紧凑构造器、with 方法
- 能写 sealed interface + 三类子类标注
- 能写 switch 表达式与模式标签、守卫、yield
- 能说出"漏分支编译错误"依赖的两个前提:sealed + 无 non-sealed
- 能用 sealed+record+switch 写出表达式树求值器
- 能写出 Optional 的 map/filter/flatMap 链与三种 orElse 终端
- 能对照 C 的 union/enum/NULL 说明类型安全差异

自测题

Q 1. record 与"手写 POJO + 手写 equals/hashCode"相比,除了代码量,最本质的差异是什么?

✅ 参考答案

不可变性。record 字段全 final,类全 final,没有 setter,equals/hashCode 由编译器按组件生成且必然一致。手写 POJO 很容易写出"字段改了但 equals 没更新""hashCode 用了可变字段"这类一致性 bug(第10章说过的哈希契约),record 从机制上消灭了整类错误。不可变还带来线程安全与可作 Map 键的免费红利。

Q 2. 下面的代码哪里错了?

```
public sealed class A { }  
public class B extends A { }
```

✅ 参考答案

两处错误:① A 的 permits 未列出 B(且 B 不在同一源文件),编译器不认 B 是合法子类;② 即使列出,直接子类必须标注 final/sealed/non-sealed,B 什么都没标,直接编译失败。修正:同文件声明可省 permits;跨文件则 `sealed class A`

permits B 且 final class B extends A。

Q 3. 为什么"sealed + switch 表达式"可以不写 default?什么情况下又必须写?

✓ 参考答案

switch 表达式要求穷尽。当 switch 的目标类型是 sealed 且 permits 列表完备、所有直接子类都被模式覆盖、且没有 non-sealed 子类时,编译器能静态证明穷尽,于是允许省略 default。一旦有任何 non-sealed 分支(或目标不是 sealed,如 Object),编译器无法证明穷尽,必须写 default(或 null 分支)。这是"类型系统穷尽性"的直观体现。

Q 4. Optional 的空盒子上调用 map 会发生什么?flatMap 与 map 的区别?

✓ 参考答案

空盒子上调用 map:不执行函数,直接返回空 Optional(短路,链式安全)。map 的函数返回普通值,Optional 自动包装;flatMap 的函数返回 Optional 本身,用于"函数内部已是 Optional 查找"的场景,避免产生 Optional<Optional<T>> 嵌套。举例:getUser(id).flatMap(u -> findOrder(u.orderId()))——findOrder 返回 Optional,若用 map 会得到嵌套。

Q 5. 用 ADT 设计一个"支付结果":成功(带金额)、失败(带原因)。写出声明与处理代码。

✓ 参考答案

```
public sealed interface PayResult permits Ok, Fail { }

public record Ok(int amount) implements PayResult { }

public record Fail(String reason) implements PayResult { }

public static void handle(PayResult r) {
    switch (r) {
        case Ok ok -> System.out.println("支付成功: " + ok.amount() + " 元");
        case Fail f -> System.out.println("支付失败: " + f.reason());
    }
}
```

关键点:sealed 保证只有两种结果;switch 无 default 但编译通过(穷尽);record 给每个结果配载荷;新增第三种结果时,编译器强制所有 switch 补分支——"修改一处,编译器帮你找全所有处理点"。

Q 6. 一个方法要从配置里读端口号,可能没有配置、可能非法。用 Optional 怎么设计?

✓ 参考答案

```
public Optional<Integer> portFrom(String cfg) {
    if (cfg == null) {
        return Optional.empty(); // 没有配置
    }
    try {
        int p = Integer.parseInt(cfg);
        return (p >= 1 && p <= 65535) ? Optional.of(p) : Optional.empty();
    } catch (NumberFormatException e) {
        return Optional.empty(); // 非法配置
    }
}

// 调用方:
int port = portFrom(config)
    .orElse(8080); // 兜底默认端口
```

要点:用 Optional 表达"可能没有",调用方三选一——默认值(orElse)、抛异常(orElseThrow)、忽略(ifPresent)。不要返回 null,不要抛异常表达"没有"(那是异常,不是缺失)。

章末实验题:JSON 迷你求值器与结算系统

实验 14:用 ADT 实现迷你表达式求值器 + 订单结算

实验目标:综合运用本章全部特性,实现两个递进任务:① 迷你表达式求值器(支持加减乘除与取负),要求扩展一种新运算后,编译器强制所有处理点更新;② 订单结算系统,用 record + Optional 处理"优惠券可能不存在/已过期"。

实验要求:

- 任务 1:用 sealed interface Expr 建模表达式,至少包含 Num / Neg / Add / Mul 四种变体(覆盖 §14.2 sealed)
- 任务 2:每个变体用 record 携带载荷(覆盖 §14.1 record)
- 任务 3:用 switch 表达式 + 模式匹配写 eval,不得使用 default(覆盖 §14.3 穷尽性)
- 任务 4:再增加 Div 变体(除零抛异常),观察所有 switch 的编译错误并逐个修复(覆盖 §14.4 编译器哨兵)
- 任务 5:实现 print(Expr) 输出中缀表达式(覆盖 §14.3 yield 多语句分支)
- 任务 6:结算系统:Order 是 record;findCoupon 返回 Optional<Coupon>;优惠券有 rate(折扣率)与 expiry;过期视为不存在(覆盖 §14.5 Optional 链)
- 任务 7:结算方法用 map/filter/orElse 组合,总价计算正确(覆盖 §14.5 链式)
- 任务 8:测试用例覆盖:合法表达式、除零、负号、无优惠券、已过期优惠券(覆盖全章验收)

实验环境:JDK 17+ 即可,单文件 javac/java 运行。

第一步 完成任务 1-4:声明 + 求值器,务必观察到"加 Div 后编译器报错"的过程

第二步 任务 5 的 print 用 yield 组装字符串

第三步 任务 6-7:Coupon 是 record(rate, expiry);findCoupon 返回 Optional;结算 = 原价 * 折扣率

参考答案要点

```

import java.time.LocalDate;
import java.util.Optional;

// ===== 任务 1-5:表达式求值器 =====
// sealed:只允许四种变体;record:每种变体自带载荷
public sealed interface Expr permits Num, Neg, Add, Mul, Div {
}

public record Num(int value) implements Expr { }

public record Neg(Expr inner) implements Expr { }

public record Add(Expr left, Expr right) implements Expr { }

public record Mul(Expr left, Expr right) implements Expr { }

// 任务 4:新增的变体—编译器会让所有 switch 报"not exhaustive"
public record Div(Expr left, Expr right) implements Expr { }

public final class ExprUtil {
    // 任务 3/4:穷尽分发,无 default;加 Div 后此处必须先修
    public static int eval(Expr e) {
        return switch (e) {
            case Num n -> n.value();
            case Neg g -> -eval(g.inner());
            case Add a -> eval(a.left()) + eval(a.right());
            case Mul m -> eval(m.left()) * eval(m.right());
            case Div d -> {
                int r = eval(d.right());
                if (r == 0) {
                    throw new ArithmeticException("除零错误");
                }
                yield eval(d.left()) / r; // 任务 5:多语句用 yield
            }
        };
    }

    // 任务 5:输出中缀表达式
    public static String print(Expr e) {
        return switch (e) {
            case Num n -> String.valueOf(n.value());
            case Neg g -> "-" + print(g.inner()) + "";
            case Add a -> "(" + print(a.left()) + " + " + print(a.right()) + "";
            case Mul m -> "(" + print(m.left()) + " * " + print(m.right()) + "";
            case Div d -> "(" + print(d.left()) + " / " + print(d.right()) + "";
        };
    }

    public static void main(String[] args) {
        // (1 + 2) * 3 / (-2) = -4 ... 1.5 取整 -4?看运算顺序:
        // ((1+2)*3)/Neg(2) -> 9 / -2 = -4(整数除法)
        Expr e = new Div(new Mul(new Add(new Num(1), new Num(2)),
            new Num(3)), new Neg(new Num(2)));
        System.out.println(print(e) + " = " + eval(e));
    }
}

// ===== 任务 6-7:订单结算系统 =====
record Coupon(double rate, LocalDate expiry) {
    boolean validOn(LocalDate day) {
        return !expiry.isBefore(day); // 未过期
    }
}

record Order(int amount, String couponCode) { }

public final class Checkout {
    // 模拟数据库:按优惠码查券,可能不存在

```

```

static Optional<Coupon> findCoupon(String code) {
    if (code == null || code.isBlank()) {
        return Optional.empty();
    }
    if (code.equals("SAVE20")) {
        return Optional.of(new Coupon(0.8, LocalDate.of(2026, 12, 31)));
    }
    if (code.equals("OLD10")) {
        return Optional.of(new Coupon(0.9, LocalDate.of(2025, 1, 1))); // 已过期
    }
    return Optional.empty();
}

// 结算:无券/过期券 → 原价;有效券 → 按折扣率
static int settle(Order order, LocalDate today) {
    return findCoupon(order.couponCode())
        .filter(c -> c.validOn(today)) // 过期即视为不存在
        .map(c -> (int) (order.amount() * c.rate()))
        .orElse(order.amount()); // 无券:原价
}

public static void main(String[] args) {
    LocalDate today = LocalDate.of(2026, 8, 26);
    System.out.println(settle(new Order(100, "SAVE20"), today)); // 80
    System.out.println(settle(new Order(100, "OLD10"), today)); // 100:过期
    System.out.println(settle(new Order(100, null), today)); // 100:无券
    System.out.println(settle(new Order(100, "NOPE"), today)); // 100:未知码
}
}

```

设计说明:任务 4 是本章最有教育意义的时刻——给 sealed 体系新增一个变体,编译器把所有 switch 的遗漏点全部找出来,这正是"类型驱动的重构"。任务 6-7 的 Optional 链(filter 过滤过期券 → map 计算折扣价 → orElse 兜底原价)是生产中最常见的三件套组合。扩展方向:给 Expr 增加"化简"方法(把 Add(Num 0, x) 化简为 x),再体验一次编译器哨兵;结算可加"满减券"变体,体会 record 组合扩展。

全书写完。14 章从数据类型到类型系统进阶,你已具备生产级 Java 能力。建议回顾:第5章内存、第6章面向对象、第9章并发、第14章类型系统——这四章是 Java 与 C 思维差异最大的地方。